07 — Compute and Feasibility Memo

Interpretable Machine Learning for Discovering Hidden Regulatory Switches in Non-Coding DNA

Field	Value
Document ID	CFM-001
Version	1.0
Status	Active
Created	2026-06-07
Related docs	Charter, Technical Design, Modeling Roadmap

1. Why This Project Is Feasible

1.1 Core Feasibility Arguments

Argument	Supporting evidence
The classification task is well-scoped	Binary classification of non-coding regions as regulatory vs. non-regulatory is a well-studied problem with established baselines (gkmSVM, DeepSEA, Basset). It does not require novel theoretical breakthroughs.
The data is publicly available	ENCODE provides pre-curated regulatory annotations (cCREs) freely downloadable. No proprietary data, lab work, or access negotiations required.
Classical ML models are sufficient for baselines	Logistic regression, random forests, and gradient-boosted trees on k-mer features have been shown to achieve AUROC 0.75–0.85 on regulatory classification tasks. These models train in minutes on CPUs.
Shallow CNNs are lightweight	A 2-layer CNN with ~200K–500K parameters trains in < 90 minutes on a single Kaggle GPU (T4). This is well within resource limits.
Interpretability tools are mature	SHAP, scikit-learn feature importance, Captum saliency — all are stable, well-documented Python libraries that run on standard hardware.
The sequence length is manageable	1 kb sequences with one-hot encoding produce (1000 × 4) arrays — 4 KB per sample. 100K samples = ~400 MB. Easily fits in Kaggle RAM.

Kaggle infrastructure is sufficient	Kaggle provides free GPU access (T4/P100), 16 GB RAM, ~20 GB disk — enough for all Phase 1–2 work and limited Phase 3 exploration.

1.2 Comparison to Infeasible Alternatives

Project type	Typical compute	Feasible here?
This project (interpretable regulatory prediction)	1 CPU/GPU session, 2–4 hours	✓ Yes
Training DeepSEA-scale CNN from scratch	1 GPU, several hours	✓ Feasible but not the primary approach
Fine-tuning DNABERT-2 (117M params) on small data	1 GPU, 1–4 hours (limited fine-tuning)	✓ Feasible in Phase 3
Training Enformer from scratch (249M params, 196 kb inputs)	Multi-GPU/TPU, days–weeks	✗ Not feasible
Training Nucleotide Transformer (2.5B params)	128+ GPUs, weeks	✗ Not feasible
Training HyenaDNA-large (6.6M–1.6B params, 1M+ bp)	Multi-GPU, days	✗ Not feasible (larger versions)
Pretraining a genomic foundation model	Institutional HPC, $10K–$100K+ compute	✗ Not feasible

---

2. Why Full Genomic Transformer Training Is Not Feasible Initially

2.1 Compute Requirements of Genomic Foundation Models

Large transformer-based genomic models require substantial computational resources:

Model	Parameters	Training hardware	Training duration	Estimated cost
DNABERT (2021)	~110M	8× V100 GPUs	Multiple days	~$1K–$5K
Nucleotide Transformer (2023)	50M–2.5B	Up to 128× A100 GPUs	Days–weeks	~$10K–$100K+
HyenaDNA (2023)	1.6M–1.6B	Multi-GPU (A100s)	Days	~$1K–$50K+
Enformer (2021)	249M	TPU v3 pod	Days	~$10K+
DNABERT-2 (2023)	~117M	Multi-GPU (A100s)	Days	~$5K+

2.2 What Kaggle Cannot Provide for Foundation Model Training

Requirement	Kaggle reality	Gap
Multi-GPU	Single GPU (T4 or P100)	Cannot parallelize across GPUs

GPU memory ≥ 40 GB	16 GB VRAM (T4)	Cannot fit large batch sizes or long sequences
Continuous training for days	Max ~12 hr session, 30 hr/week GPU quota	Cannot sustain multi-day training
High-speed interconnect	Not applicable (single machine)	No distributed training
Large disk I/O	~20 GB scratch	Insufficient for large pretraining corpora

2.3 What Kaggle CAN Provide

Capability	Detail
Single-GPU training	Sufficient for shallow CNNs (200K–2M params), fine-tuning small models
CPU-based ML training	Sufficient for logistic regression, random forests, XGBoost on tabular features
Pretrained model inference	Load a pretrained checkpoint (if ≤ 16 GB) and extract embeddings
Limited fine-tuning	Fine-tune last few layers of a small pretrained model (≤ 200M params)
Feature extraction from pretrained	Forward pass through frozen pretrained model to get embeddings
SHAP / saliency computation	CPU or GPU; well within limits for 1000–10,000 samples
Data preprocessing	Sequence extraction, k-mer computation, one-hot encoding — all CPU-bound, feasible

---

3. Expected Compute Bottlenecks

Bottleneck	Phase	Severity	Mitigation
k-mer computation for k=6	Phase 1	Medium	Use sparse matrices; compute in batches; cache results
SHAP computation for large datasets	Phase 1–2	Medium	Subsample to 1K–5K samples for SHAP; use TreeExplainer for tree-based models (faster)
CNN training on full dataset	Phase 2	Medium	Use mini-batches (64–256); early stopping; limit to 50 epochs
Pretrained model loading	Phase 3	High	Use smallest available checkpoint; float16 inference; may need to process in batches

Saliency map computation for full test set	Phase 2–3	Low-Medium	Compute for representative subset only (100–1000 samples)
Reference genome loading	Phase 0–1	Low	Load per-chromosome; use indexed FASTA (pyfaidx)
Multiple cell-type experiments	Phase 2	Medium	Run sequentially; cache intermediate features

---

4. Mitigation Strategies

4.1 Memory Management

Strategy	Implementation
Per-chromosome genome loading	Use `pyfaidx` for indexed random access; never load full genome into RAM
Sparse k-mer matrices	Use `scipy.sparse.csr_matrix` for k ≥ 5
Float32 precision	Use `np.float32` everywhere; avoid float64
Batch processing	Process sequences in chunks of 10K; write features to disk
Garbage collection	Explicit `del` + `gc.collect()` after large operations
Lazy data loading (CNN)	Use PyTorch `DataLoader` with on-disk dataset

4.2 Training Time Management

Strategy	Implementation
Early stopping	Stop CNN training if validation loss hasn't improved in 10 epochs
Learning rate scheduling	ReduceLROnPlateau to converge faster
Limit hyperparameter search	Manual tuning or small random search (20 trials) instead of grid search
Cache features	Compute features once, save to `.npz`, reload for model training
Use LightGBM over XGBoost	LightGBM is often 2–5× faster for similar accuracy

4.3 Kaggle Quota Management

Strategy	Detail
CPU for classical ML	Use CPU runtime for LogReg, RF, XGBoost — doesn't consume GPU quota

GPU only for CNN + saliency	Switch to GPU runtime only when training neural models or computing gradients
Checkpoint and resume	Save model checkpoints every 5 epochs; restart from checkpoint if session expires
Batch notebook runs	Run data preprocessing and feature computation in one session; model training in another
Pre-compute features locally	If team has any local GPU, pre-compute and upload features as Kaggle dataset

---

5. Cost-Conscious Experimentation Strategy

5.1 Experiment Prioritization

Run the cheapest experiments first to validate assumptions before investing GPU time.

```
Priority 1 (CPU, minutes):
  ■■■ LogReg on k=4 k-mers → Sanity check. Is the task learnable?
  |
Priority 2 (CPU, 5–20 min):
  ■■■ RF / XGBoost on full features → Best interpretable baseline.
  |
Priority 3 (CPU, 10–30 min):
  ■■■ SHAP analysis → Feature importance for best classical model.
  |
Priority 4 (GPU, 30–90 min):
  ■■■ Shallow CNN → Does sequence-level learning add value?
  |
Priority 5 (GPU, 1–3 hr):
  ■■■ CNN saliency + filter analysis → Interpretability from DL.
  |
Priority 6 (GPU, 2–4 hr):
  ■■■ Pretrained embedding extraction + classifier → Foundation model test.
```

5.2 Fail-Fast Checkpoints

Checkpoint	If result is bad...	Action
LogReg AUROC < 0.60	Task may not be learnable from k-mers alone	Check data pipeline; verify labels; try different k
RF AUROC < 0.70	Features may be insufficient	Add motif features; try different negative sampling
CNN AUROC ≤ RF AUROC	CNN doesn't help for this data	Stay with tree-based models; don't invest in deeper DL
Pretrained embedding doesn't improve	Embeddings may not capture regulatory info at this scale	Stay with CNN or tree-based; document negative result

5.3 Compute Budget (Monthly)

Resource	Monthly budget	How used

Kaggle GPU hours	~30 hours/week × 4 = 120 hr/month	CNN training, saliency, pretrained inference
Kaggle CPU hours	Effectively unlimited	Classical ML, data processing, feature computation
Kaggle disk	~20 GB per notebook	Dataset storage, feature caches, model artifacts
Local machine	Variable (team laptops)	Code development, testing, small experiments
Cloud compute (if any)	$0 initially; $50–100/month if funded	Only for Phase 3+ if pretrained models exceed Kaggle limits

---

6. Platform Comparison

Platform	GPU	RAM	Disk	Session limit	Cost	Best for
Kaggle (recommended)	T4 (16 GB) or P100 (16 GB)	16 GB (GPU), 30 GB (CPU)	~20 GB	12 hr; 30 hr/week GPU	Free	All Phase 0–2 work
Google Colab (free)	T4 (15 GB)	12.7 GB	~100 GB (temp)	Variable; may disconnect	Free	Backup option
Google Colab Pro	V100 or A100	Up to 52 GB	~100 GB	Longer sessions	$12/month	Phase 3 if needed
Local laptop (no GPU)	None	8–16 GB	Variable	Unlimited	$0	Development, testing
AWS/GCP spot instances	V100/A100	Configurable	Configurable	Unlimited	$0.50–3/hr	Phase 3–4 if funded

Recommendation: Start with Kaggle for all initial work. Only consider Colab Pro or cloud instances if Phase 3 pretrained experiments require more memory.

---

7. Scaling Scenarios

7.1 If the project needs more compute in the future:

Scenario	Trigger	Recommended action
Pretrained model doesn't fit in Kaggle GPU memory	Loading DNABERT-2 or larger model	Use smallest checkpoint; try float16; consider Colab Pro
Need to run > 30 GPU hours/week	Many CNN experiments or hyperparameter sweeps	Spread across team members' Kaggle accounts; use CPU where possible
Need persistent experiment tracking server	Too many experiments for file-based logging	Set up free-tier W&B account

Need to process more cell types	Expanding from 2 to 10+ cell types	Batch processing; pre-compute features locally

7.2 What would require leaving Kaggle entirely:

- Training a model with > 500M parameters (not planned).
- Processing sequences > 10 kb at scale (not planned in initial phases).
- Real-time inference serving (not planned initially).
- Multi-GPU distributed training (not planned).

None of these are in scope for the initial project build.

End of Document 07 — Compute and Feasibility Memo
Next: 08 — Experiment Tracking & MLOps Lite